Basic Input/Output Operations

Accessing I/O Devices

I/O

- The data on which the instructions operate are not necessarily already stored in memory.
- Data need to be transferred between processor and outside world (disk, keyboard, etc.)
- I/O operations are essential, the way they are performed can have a significant effect on the performance of the computer.

Accessing I/O devices

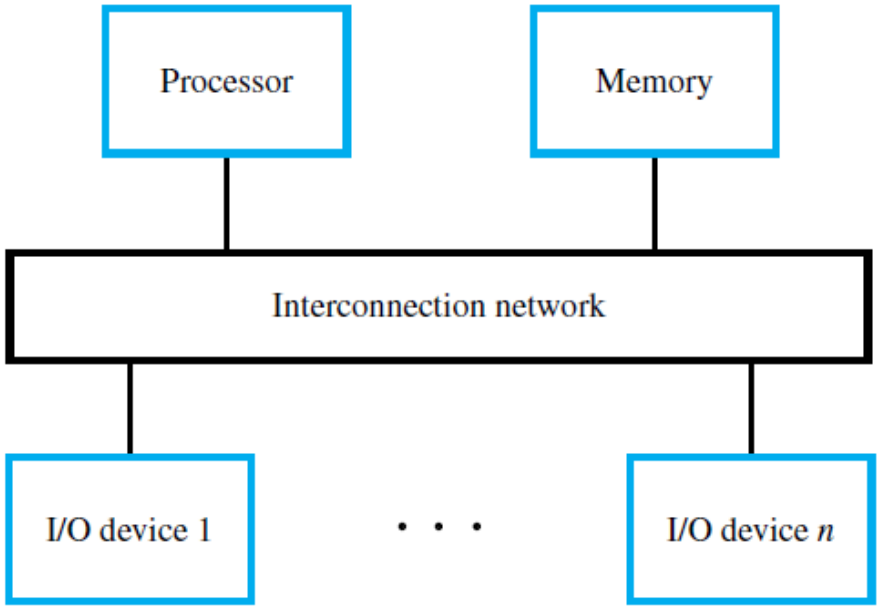


Figure 3.1 A computer system.

- *Multiple I/O devices may be connected to the processor and the memory via a bus.*
- *Bus consists of three sets of lines to carry address, data and control signals.*
- *Each I/O device is assigned an unique address.*
- *To access an I/O device, the processor places the address on the address lines.*
- *The device recognizes the address, and responds to the control signals.*

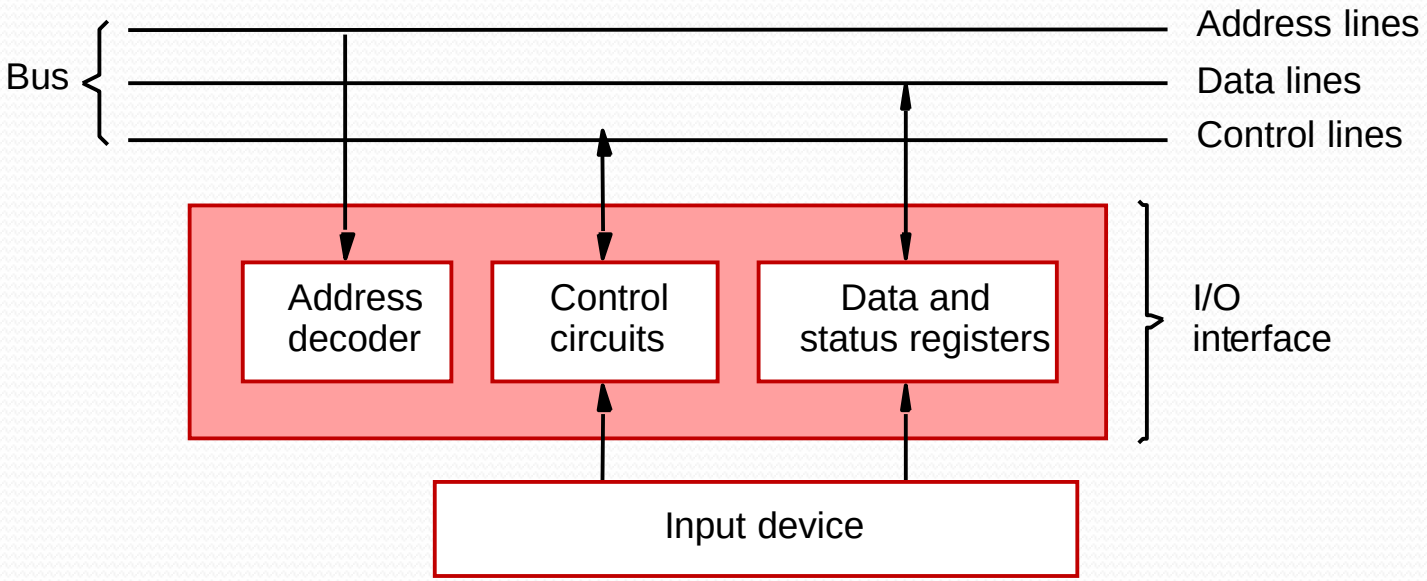
Accessing I/O devices (contd..)

- I/O devices and the memory may share the same address space:
  - Memory-mapped I/O.
  - Any machine instruction that can access memory can be used to transfer data to or from an I/O device.

    Load R2, DATAIN ; DATAIN is the address of an i/p device register

    Store R2, DATAOUT ; DATAOUT is the address of an o/p device register
  - Simpler software.
- I/O devices and the memory may have different address spaces:
  - Special instructions to transfer data to and from I/O devices.
  - I/O devices may have to deal with fewer address lines.
  - I/O address lines need not be physically separate from memory address lines.
  - In fact, address lines may be shared between I/O devices and memory, with a control signal to indicate whether it is a memory address or an I/O address.

Accessing I/O devices (contd..)



- *I/O device is connected to the bus using an I/O interface circuit which has:*
  - *Address decoder, control circuit, and data and status registers.*
- *Address decoder decodes the address placed on the address lines thus enabling the device to recognize its address.*
- *Data register holds the data being transferred to or from the processor.*
- *Status register holds information necessary for the operation of the I/O device.*
- *Data and status registers are connected to the data lines, and have unique addresses.*
- *I/O interface circuit coordinates I/O transfers.*

Accessing I/O devices (contd..)

- Recall that the rate of transfer to and from I/O devices is slower than the speed of the processor. This creates the need for mechanisms to synchronize data transfers between them.
- Program-controlled I/O:
  - Processor repeatedly monitors a status flag to achieve the necessary synchronization.
  - Processor polls the I/O device.
- Two other mechanisms used for synchronizing data transfers between the processor and memory:
  - Interrupts.
  - Direct Memory Access.

Program-Controlled I/O Example

- Read in character input from a keyboard and produce character output on a display screen.
- Rate of data transfer (keyboard, display, processor)
- Difference in speed between processor and I/O device creates the need for mechanisms to synchronize the transfer of data.
- A solution: on output, the processor sends the first character and then waits for a signal from the display that the character has been received. It then sends the second character. Input is sent from the keyboard in a similar way.

Program-Controlled I/O Example

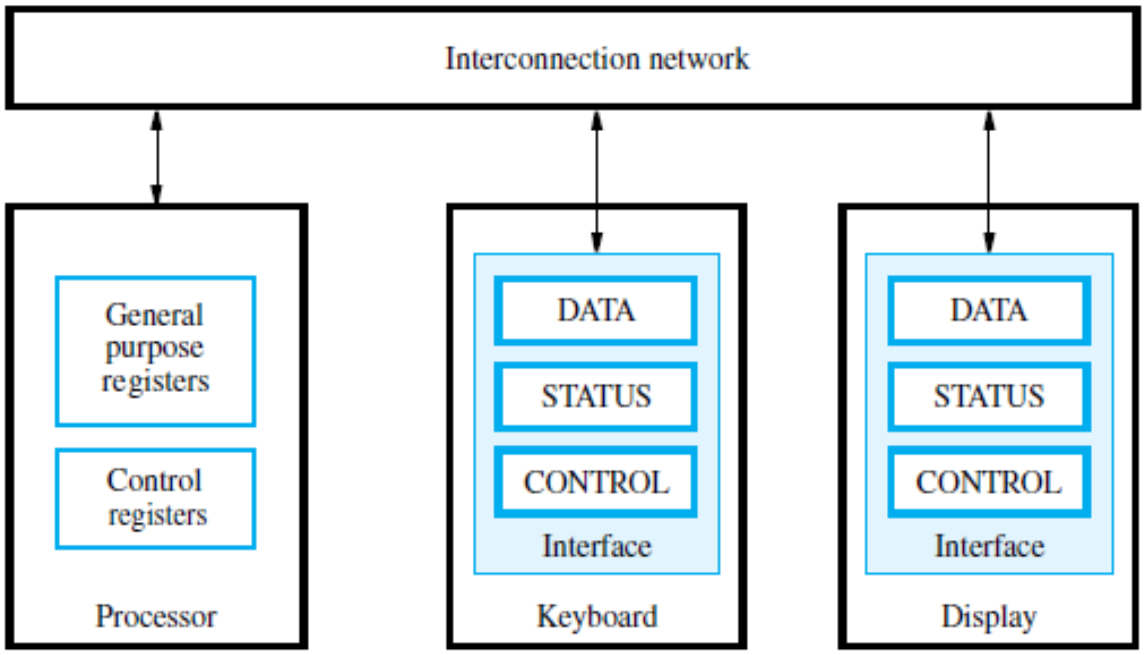
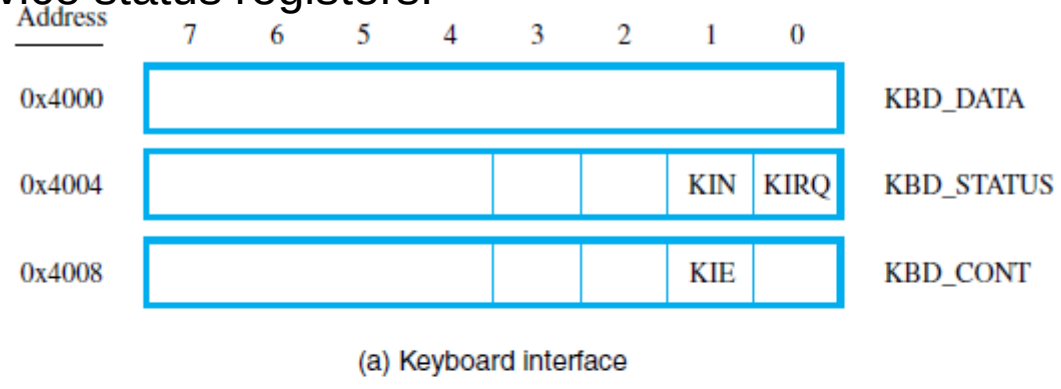


Figure 3.2 The connection for processor, keyboard, and display.

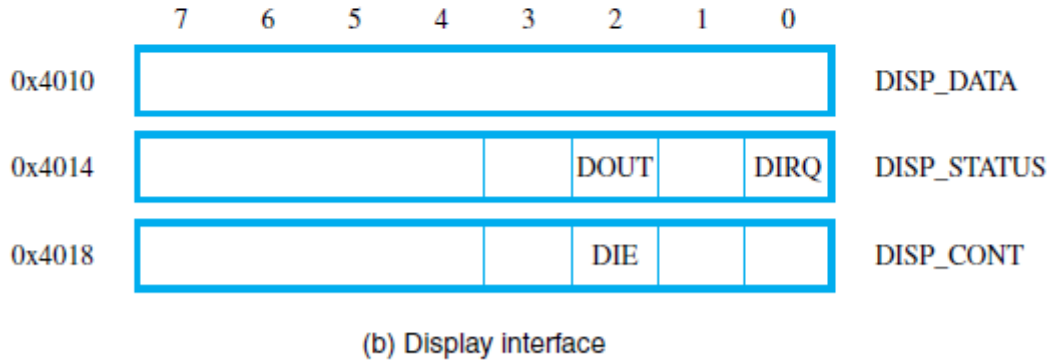
- Registers
- Flags
- Device interface

Program-Controlled I/O Example

Memory-Mapped I/O – some memory address values are used to refer to peripheral device buffer registers. No special instructions are needed. Also use device status registers.



(a) Keyboard interface



(b) Display interface

Figure 3.3 Registers in the keyboard and display interfaces.

Program-Controlled I/O Example

- Machine instructions that can check the state of the status flags and transfer data:

```
READWAIT    Read the KIN flag
            Branch to READWAIT if KIN = 0
            Transfer data from KBD_DATA to R5

WRITEWAIT   Read the DOUT
            Branch to WRITEWAIT if DOUT = 0
            Transfer data from R5 to DISP_DATA
```

Program-Controlled I/O Example

- The Read operation described above may be implemented by the RISC-style instructions:

```
READWAIT:   LoadByte          R4, KBD_STATUS
            And               R4, R4, #2
            Branch_if_[R4]=0  READWAIT
            LoadByte          R5, KBD_DATA
```

Similarly, the Write operation may be implemented as:

```
WRITEWAIT:  LoadByte          R4, DISP_STATUS
            And               R4, R4, #4
            Branch_if_[R4]=0  WRITEWAIT
            StoreByte         R5, DISP_DATA
```

Program-Controlled I/O Example

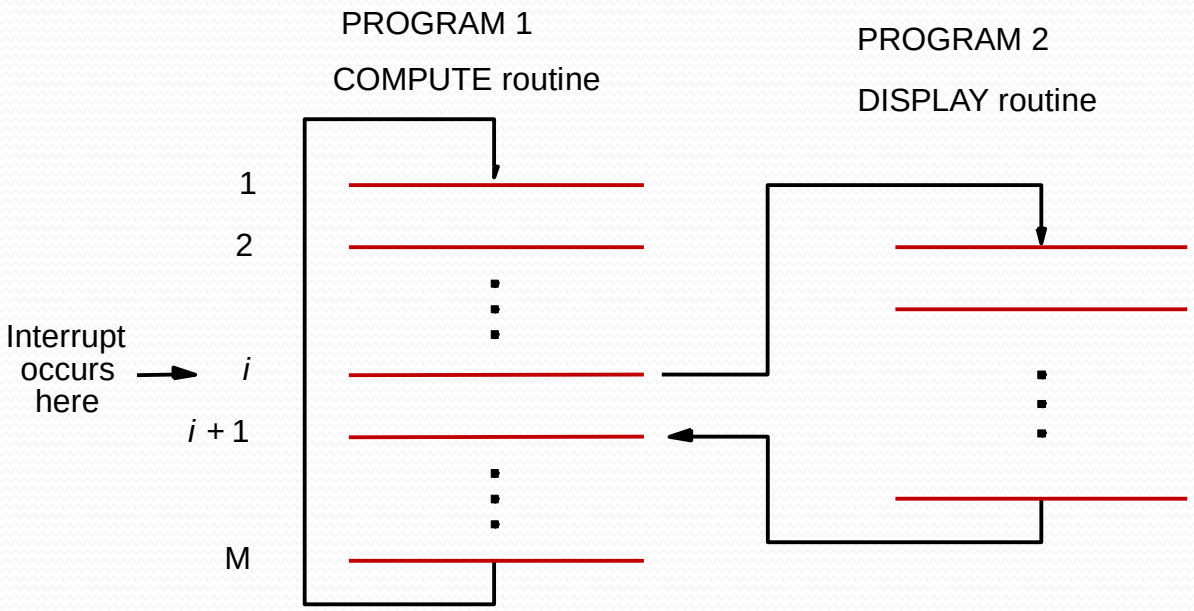
- Any drawback of program controlled I/O in terms of efficiency?
  - requires continuous involvement of the processor in the I/O activities
  - Two wait loops, processor waiting for the key to be pressed or for the display to become available→processor execution time is wasted
- Alternate solution?
  - Interrupt

Interrupts

Interrupts

- In program-controlled I/O, when the processor continuously monitors the status of the device, it does not perform any useful tasks.
- An alternate approach would be for the I/O device to alert the processor when it becomes ready.
  - Do so by sending a hardware signal called an interrupt to the processor.
  - At least one of the bus control lines, called an interrupt-request line is dedicated for this purpose.
- Processor can perform other useful tasks while it is waiting for the device to be ready.

Interrupts (contd..)



- *Consider the task that requires continuous extensive computations to be performed and the results to be displayed on display device.*
- *Processor is executing the instruction located at address i when an interrupt occurs.*
- *Routine executed in response to an interrupt request is called the interrupt-service routine.*
- *When an interrupt occurs, control must be transferred to the interrupt service routine.*
- *But before transferring control, the current contents of the PC (i+1), must be saved in a known location.*
- *This will enable the return-from-interrupt instruction to resume execution at i+1.*
- *Return address, or the contents of the PC are usually stored on the processor stack.*

Interrupts (contd..)

- Treatment of an interrupt-service routine is very similar to that of a subroutine.
- However there are significant differences:
  - A subroutine performs a task that is required by the calling program.
  - Interrupt-service routine may not have anything in common with the program it interrupts.
  - Interrupt-service routine and the program that it interrupts may belong to different users.
  - As a result, before branching to the interrupt-service routine, not only the PC, but other information such as condition code flags, and processor registers used by both the interrupted program and the interrupt service routine must be stored.
  - This will enable the interrupted program to resume execution upon return from interrupt service routine.

Interrupts (contd..)

- Saving and restoring information can be done automatically by the processor or explicitly by program instructions.
- Saving and restoring registers involves memory transfers:
  - Increases the total execution time.
  - Increases the delay between the time an interrupt request is received, and the start of execution of the interrupt-service routine. This delay is called interrupt latency.
- In order to reduce the interrupt latency, most processors save only the minimal amount of information:
  - This minimal amount of information includes Program Counter and processor status registers.
- Any additional information that must be saved, must be saved explicitly by the program instructions at the beginning of the interrupt service routine.

Interrupts (contd..)

- Some computers provide two types of interrupts. One saves all register contents, and the other does not.
- A particular I/O device may use either type, depending upon its response time requirements.
- Another interesting approach is to provide duplicate sets of processor registers.
  - In this case, a different set of registers can be used by the interrupt-service routine, thus eliminating the need to save and restore registers. The duplicate registers are sometimes called the shadow registers.
- Interrupt concept is used in used in operating systems and in many control applications where processing of certain routines must be accurately timed relative to external events (Real-time processing).

Interrupts (contd..)

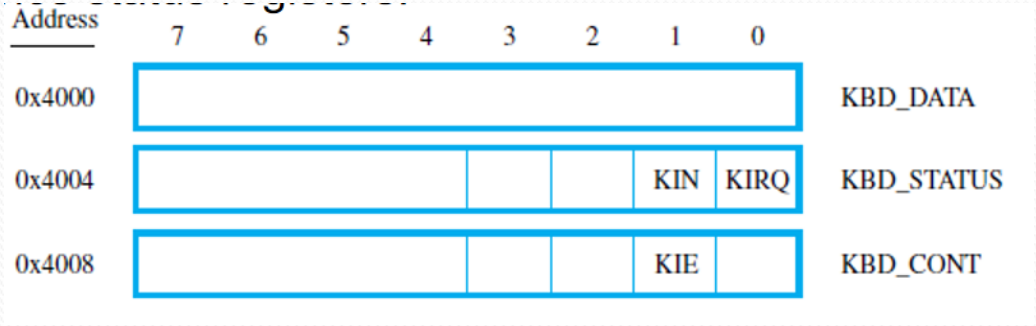
- When a processor receives an interrupt-request, it must branch to the interrupt service routine.
- It must also inform the device that it has recognized the interrupt request.
- This can be accomplished in two ways:
  - Some processors have an explicit interrupt-acknowledge control signal for this purpose.
  - In other cases, the data transfer that takes place between the device and the processor can be used to inform the device.

Interrupts (contd..)

- Interrupt-requests interrupt the execution of a program, and may alter the intended sequence of events:
  - Sometimes such alterations may be undesirable, and must not be allowed.
  - For example, the processor may not want to be interrupted by the same device while executing its interrupt-service routine.
- Processors generally provide the ability to enable and disable such interruptions as desired. It should be available at the Device end also.
- The processor has a status register (PS), which contains information about its current state of operation. One bit, IE, of this register be assigned for enabling/disabling interrupts.
- To avoid interruption by the same device during the execution of an interrupt service routine:
  - First instruction of an interrupt service routine can be Interrupt-disable.
  - Last instruction of an interrupt service routine can be Interrupt-enable.

Interrupts (contd..)

- The interface of an I/O device includes a control register that contains the information that governs the mode of operation of the device.
- One bit in this register may be dedicated to interrupt control.
- The I/O device is allowed to raise interrupt requests only when this bit is set to 1.



Interrupts (contd..)

- To summarize the sequence of events involved in handling an interrupt request from a single device.
  - The device raises an interrupt request.
  - The processor interrupts the program currently being executed and saves the contents of the PC and PS registers.
  - Interrupts are disabled by clearing the IE bit in the PS to 0.
  - The action requested by the interrupt is performed by the interrupt-service routine, during which time the device is informed that its request has been recognized, and in response, it deactivates the interrupt-request signal.
  - Upon completion of the interrupt-service routine, the saved contents of the PC and PS registers are restored (enabling interrupts by setting the IE bit to 1), and execution of the interrupted program is resumed.

Interrupts (contd..)

- Multiple I/O devices may be connected to the processor and the memory via a bus. Some or all of these devices may be capable of generating interrupt requests.
  - Each device operates independently, and hence no definite order can be imposed on how the devices generate interrupt requests?
- How does the processor know which device has generated an interrupt?
- How does the processor know which interrupt service routine needs to be executed?
- When the processor is executing an interrupt service routine for one device, can other device interrupt the processor?
- If two interrupt-requests are received simultaneously, then how to break the tie?

Interrupts (contd..)

- Consider a simple arrangement where all devices send their interrupt-requests over a single control line in the bus.
- When the processor receives an interrupt request over this control line, how does it know which device is requesting an interrupt?
- This information is available in the status register of the device requesting an interrupt:
  - The status register of each device has an *IRQ* bit which it sets to 1 when it requests an interrupt.
- Interrupt service routine can poll the I/O devices connected to the bus. The first device with *IRQ* equal to 1 is the one that is serviced.
- Polling mechanism is easy, but time consuming to query the status bits of all the I/O devices connected to the bus.

Interrupts (contd..)

- Vectored Interrupts: The device requesting an interrupt may identify itself directly to the processor.
  - Device can do so by sending a special code (4 to 8 bits) the processor over the bus.
  - Code supplied by the device may represent a part of the starting address of the interrupt-service routine.
  - The remainder of the starting address is obtained by the processor based on other information such as the range of memory addresses where interrupt service routines are located.
- Usually the location pointed to by the interrupting device is used to store the starting address of the interrupt-service routine.

Interrupts (contd..)

- Previously, before the processor started executing the interrupt service routine for a device, it disabled the interrupts from the device.
- In general, same arrangement is used when multiple devices can send interrupt requests to the processor.
  - During the execution of an interrupt service routine of device, the processor does not accept interrupt requests from any other device.
  - Since the interrupt service routines are usually short, the delay that this causes is generally acceptable.
- However, for certain devices this delay may not be acceptable. Eg: a computer that keeps track of time of day using real-time clock
  - It may be necessary to accept an interrupt nesting from some devices (clock) during the execution of ISR.
  - Which devices can be allowed to interrupt a processor when it is executing an interrupt service routine of another device?

Interrupts (contd..)

- I/O devices are organized in a priority structure:
  - An interrupt request from a high-priority device is accepted while the processor is executing the interrupt service routine of a low priority device.
- A priority level is assigned to a processor that can be changed under program control.
  - Priority level of a processor is the priority of the program that is currently being executed.
  - When the processor starts executing the interrupt service routine of a device, its priority is raised to that of the device.
  - If the device sending an interrupt request has a higher priority than the processor, the processor accepts the interrupt request.

Interrupts (contd..)

- Processor's priority is encoded in a few bits of the processor status register.
  - Priority can be changed by instructions that write into the processor status register.
  - Usually, these are privileged instructions, or instructions that can be executed only in the supervisor mode.
  - Privileged instructions cannot be executed in the user mode.
  - Prevents a user program from accidentally or intentionally changing the priority of the processor.
- If there is an attempt to execute a privileged instruction in the user mode, it causes a special type of interrupt called as privilege exception.

Interrupts (contd..)

- **Simultaneous Interrupts:**
- When simultaneous interrupt requests arrive processor must have some means of deciding which request to service first.
- **Polling** the status registers of the I/O devices is the simplest such mechanism. In this case, priority is determined by the order in which the devices are polled.
- In **Vectored** interrupts arbitration aitration circuit selects only one device to send its interrupt vector code.

Interrupts (contd..)

- Controlling I/O device behavior:
- To ensure that interrupt requests are generated only by those I/O devices that the processor is currently willing to recognize, a mechanism is needed in the interface circuits of individual devices to control whether a device is allowed to interrupt the processor.
- KIE=1, KB can raise interrupt request
- KIRQ=1 if KB has raised the interrupt but not serviced by processor yet.

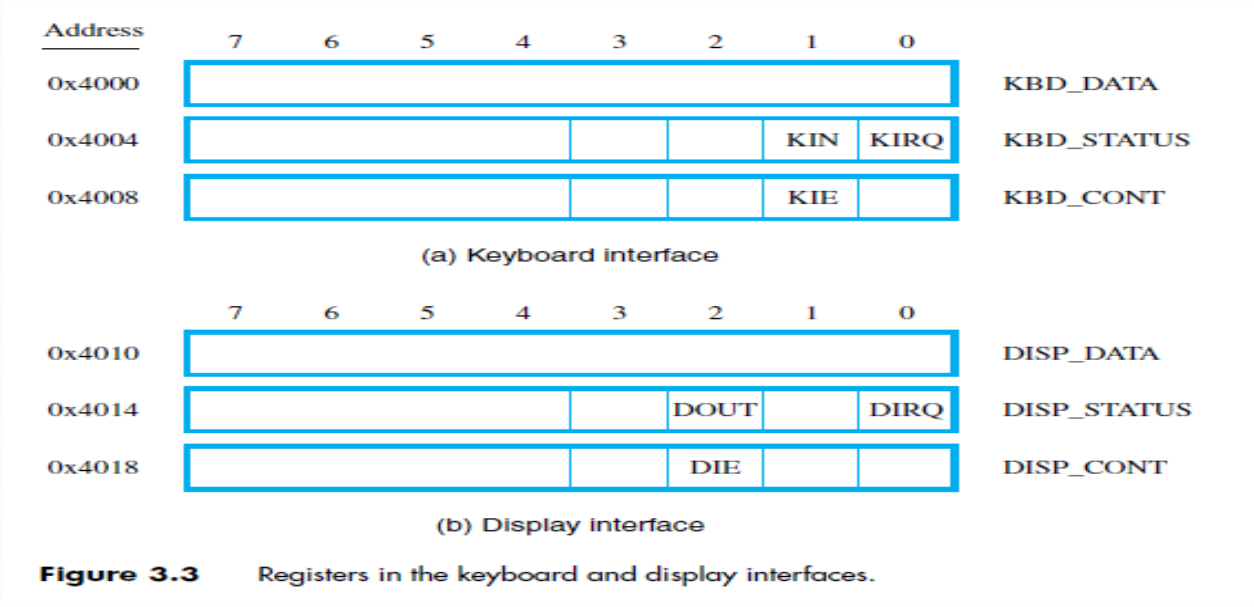


Figure 3.3 Registers in the keyboard and display interfaces.

Interrupts (contd..)

- **Processor control registers:**
- Figure 3.7 depicts one possibility, where there are four processor control registers.

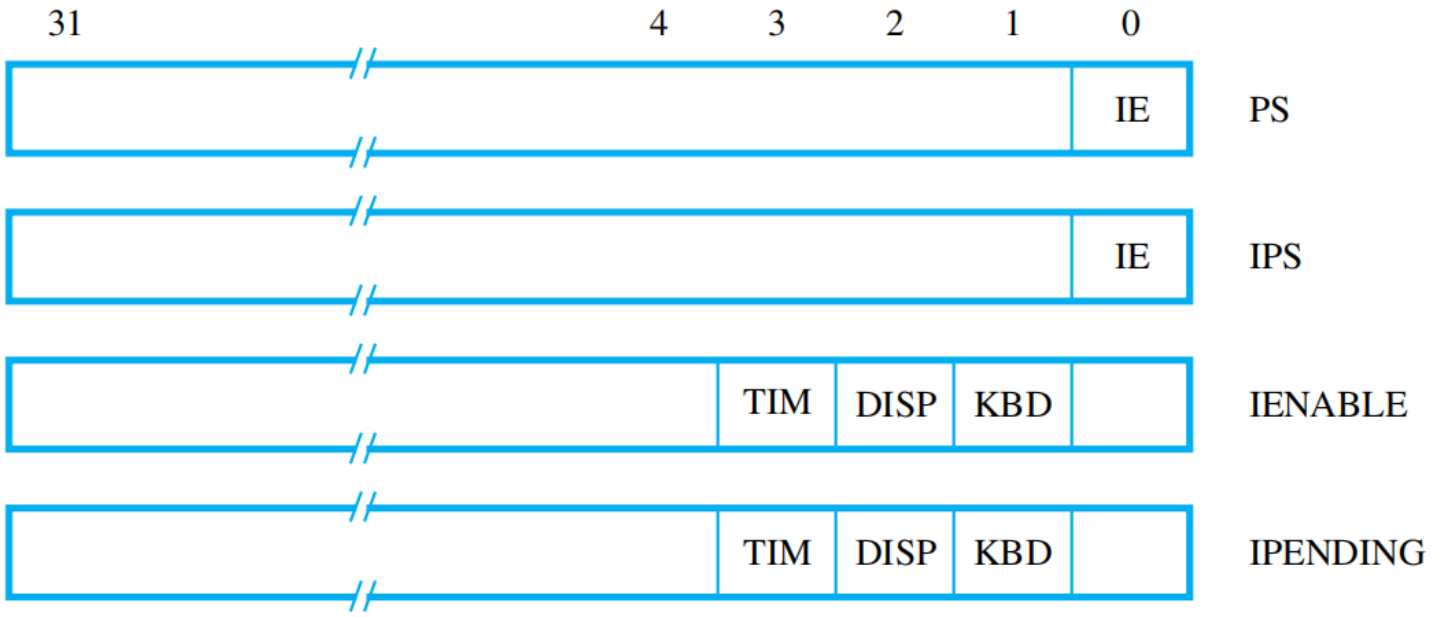


Figure 3.7 Control registers in the processor.

Interrupts (contd..)

- **Processor control registers:**
- The status register, PS, includes the interrupt-enable bit, IE, in addition to other status information.
- The IPS register is used to automatically save the contents of PS when an interrupt request is received and accepted.
- The IENABLE register allows the processor to selectively respond to individual I/O devices. When a bit is set to 1, the processor will accept interrupt requests from the corresponding device.
- The IPENDING register indicates the active interrupt requests. This is convenient when multiple devices may raise requests at the same time. Then, a program can decide which interrupt should be serviced first.
- Special instructions are provided to access these registers.

```
MoveControl R2, PS     ; R2 <-- [PS]
MoveControl IENABLE, R3      ; IENABLE <-- [R3]
```

Interrupts (contd..)

- Only those devices that are being used in a program should be allowed to generate interrupt requests.
- To control which devices are allowed to generate interrupt requests,
  - If the interrupt-enable bit in the device interface (device control register) is set to 1, then the device is allowed to generate an interrupt-request.
  - By setting the corresponding bit in IENABLE control register in processor, the processor selectively respond to individual I/O devices.
- Interrupt-enable bit in the device's interface circuit (control register)determines whether the device is allowed to generate an interrupt request.
- Interrupt-enable bit in the processor status register (PS) or the priority structure of the interrupts determines whether a given interrupt will be accepted.

Interrupts (contd..)

- A RISC-style program that reads a line of characters using interrupts, and displays the line using polling.

Exceptions

- Interrupts caused by interrupt-requests sent by I/O devices.
- Interrupts could be used in many other situations where the execution of one program needs to be suspended and execution of another program needs to be started.
- In general, the term exception is used to refer to any event that causes an interruption.
  - Interrupt-requests from I/O devices is one type of an exception.
- Other types of exceptions are:
  - Recovery from errors
  - Debugging
  - Privilege exception

Exceptions (contd..)

- Many sources of errors in a processor. For example:
  - Error in the data stored.
  - Error during the execution of an instruction.
- When such errors are detected, exception processing is initiated.
  - Processor takes the same steps as in the case of I/O interrupt-request.
  - It suspends the execution of the current program, and starts executing an exception-service routine.
- Difference between handling I/O interrupt-request and handling exceptions due to errors:
  - In case of I/O interrupt-request, the processor usually completes the execution of an instruction in progress before branching to the interrupt-service routine.
  - In case of exception processing however, the execution of an instruction in progress usually cannot be completed.

Exceptions (contd..)

- Debugger uses exceptions to provide important features:
  - Trace,
  - Breakpoints.
- Trace mode:
  - Exception occurs after the execution of every instruction.
  - Debugging program is used as the exception-service routine.
- Breakpoints:
  - Exception occurs only at specific points selected by the user.
  - Debugging program is used as the exception-service routine.

Exceptions (contd..)

- Certain instructions can be executed only when the processor is in the supervisor mode. These are called privileged instructions.
- If an attempt is made to execute a privileged instruction in the user mode, a privilege exception occurs.
- Privilege exception causes:
  - Processor to switch to the supervisor mode,
  - Execution of an appropriate exception-servicing routine.

Buses

Bus Structure

- Only one source/destination pair of units can use this bus to transfer data at any one time.
- The bus consists of three sets of lines used to carry address, data, and control signals.

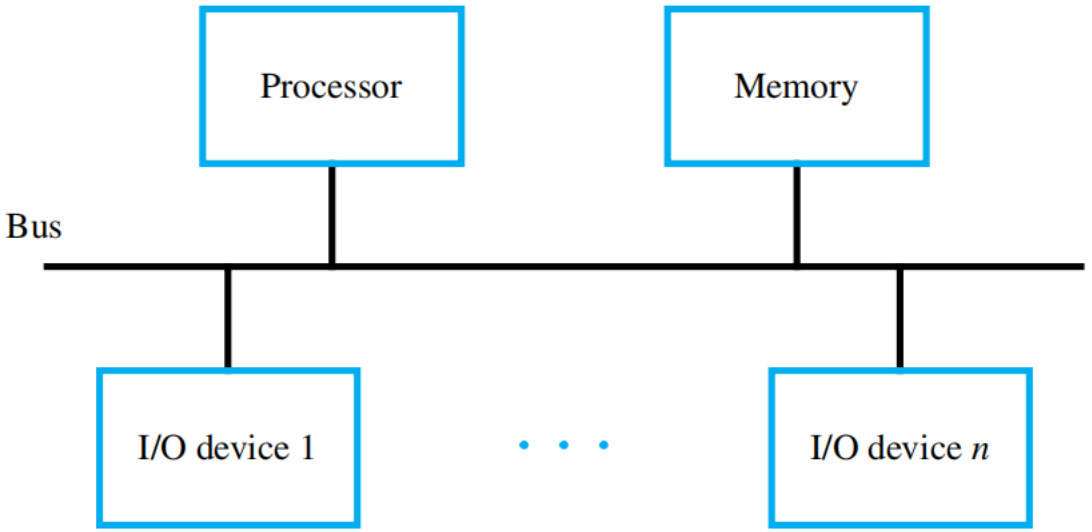


Figure 7.1 A single-bus structure.

Buses

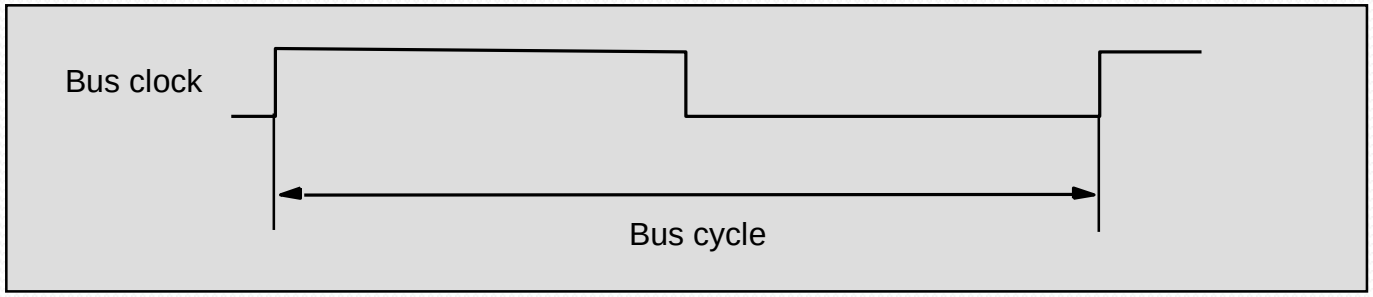
- Processor, main memory, and I/O devices are interconnected by means of a bus.
- Bus provides a communication path for the transfer of data.
  - Bus also includes lines to support interrupts and arbitration.
- A bus protocol is the set of rules that govern the behavior of various devices connected to the bus, as to when to place information on the bus, when to assert control signals, etc.

Buses (contd..)

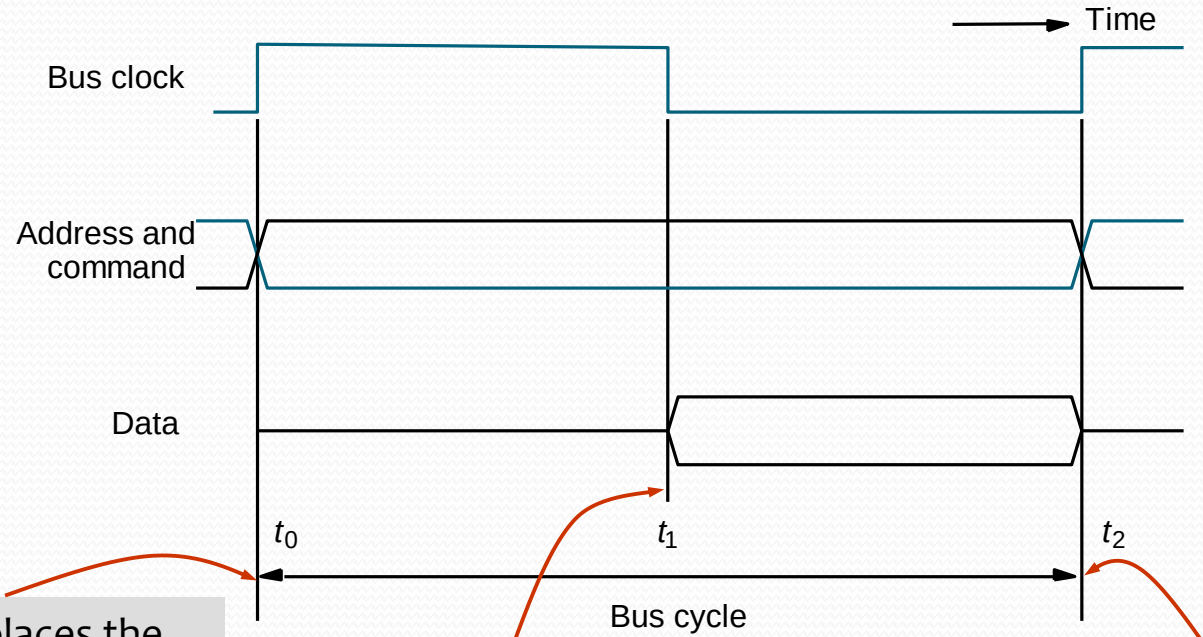
- Bus lines may be grouped into three types:
  - Data
  - Address
  - Control
- Control signals specify:
  - Whether it is a read or a write operation.
  - Required size of the data, when several operand sizes (byte, word, long word) are possible.
  - Timing information to indicate when the processor and I/O devices may place data or receive data from the bus.
- Schemes for timing of data transfers over a bus can be classified into:
  - Synchronous,
  - Asynchronous.

Synchronous bus

- On a synchronous bus, all devices derive timing information from a control line called the bus clock.
- The signal on this line has two phases: a high level followed by a low level.
- The two phases constitute a clock cycle. The first half of the cycle between the low-to-high and high-to-low transitions is often referred to as a clock pulse.



Synchronous bus (contd..)



Master places the device address and command on the bus, and indicates that it is a Read operation.

Addressed slave places data on the data lines

Master "strobes" the data on the data lines into its input buffer, for a Read operation.

- *In case of a Write operation, the master places the data on the bus along with the address and commands at time t_o.*
- *The slave strobes the data into its input buffer at time t_2.*

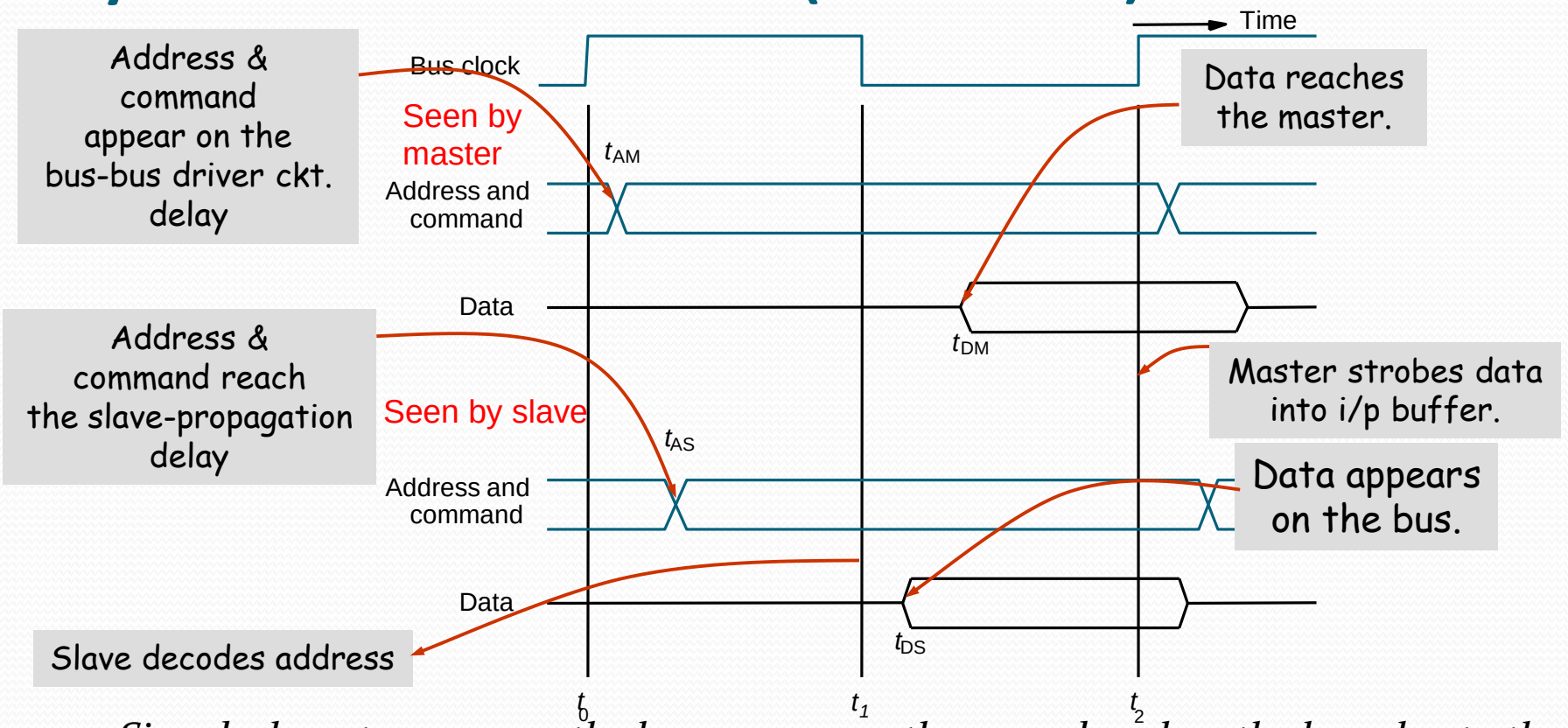
Synchronous bus (contd..)

- Once the master places the device address and command on the bus, it takes time for this information to propagate to the devices:
  - This time depends on the physical and electrical characteristics of the bus.
- Also, all the devices have to be given enough time to decode the address and control signals, so that the addressed slave can place data on the bus.
- Width of the pulse t_1 - t_0 depends on:
  - Maximum propagation delay between two devices connected to the bus.
  - Time taken by all the devices to decode the address and control signals, so that the addressed slave can respond at time t_1.

Synchronous bus (contd..)

- At the end of the clock cycle, at time t_2, the master strobes the data on the data lines into its input buffer if it's a Read operation.
  - "Strobe" means to capture the values of the data and store them into a buffer.
- When data are to be loaded into a storage buffer register, the data should be available for a period longer than the setup time of the device.
- Width of the pulse t_2 - t_1 should be longer than:
  - Maximum propagation time of the bus plus
  - Set up time of the input buffer register of the master.

Synchronous bus (contd..)



- *Signals do not appear on the bus as soon as they are placed on the bus, due to the propagation delay in the interface circuits.*
- *Signals reach the devices after a propagation delay which depends on the characteristics of the bus.*
- *t_2 - t_o -> set time for the master's i/p buffer*
- *Data must remain on the bus for some time after t_2 equal to the hold time of the buffer.*

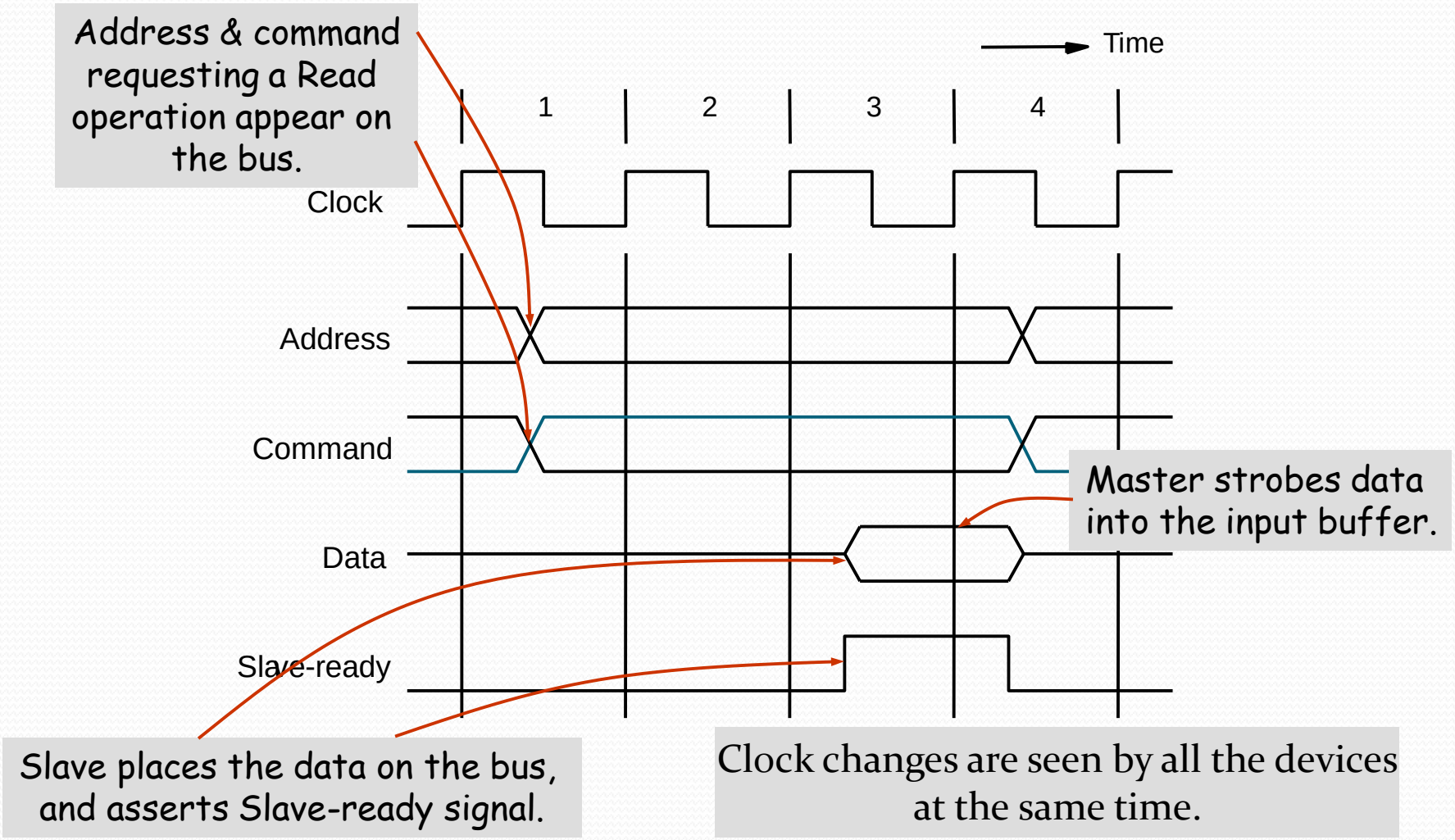
Synchronous bus (contd..)

- Data transfer has to be completed within one clock cycle.
  - Clock period $t_2 - t_0$ must be such that the longest propagation delay on the bus and the slowest device interface must be accommodated.
  - Forces all the devices to operate at the speed of the slowest device.
- Processor just assumes that the data are available at t2 in case of a Read operation, or are read by the device in case of a Write operation.
  - What if the device is actually failed, and never really responded?

Synchronous bus (contd..)

- **Multiple Cycle Transfer:**
- Most buses have control signals to represent a response from the slave.
- Control signals serve two purposes:
  - Inform the master that the slave has recognized the address, and is ready to participate in a data transfer operation.
  - Enable to adjust the duration of the data transfer operation based on the speed of the participating slaves.
- High-frequency bus clock is used:
  - Data transfer spans several clock cycles instead of just one clock cycle as in the earlier case.

Synchronous bus (contd..)



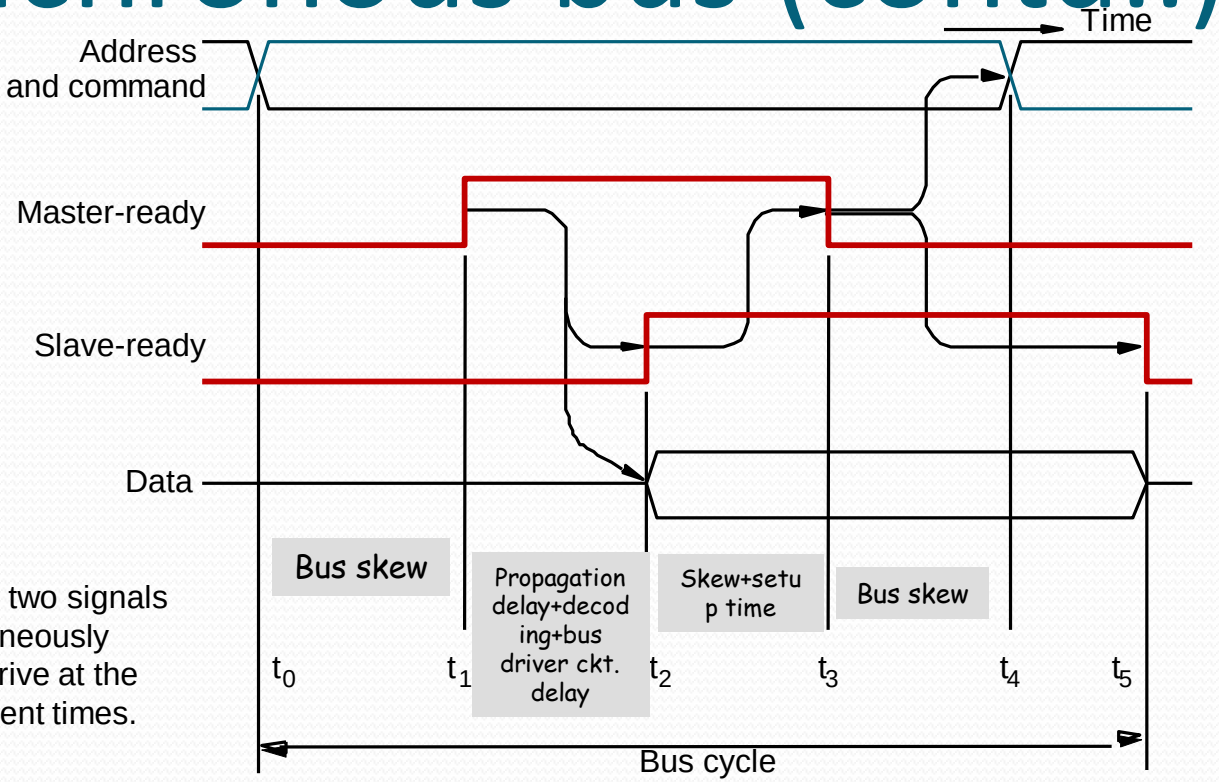
Asynchronous bus

- Data transfers on the bus is controlled by a handshake between the master and the slave.
- Common clock in the synchronous bus case is replaced by two timing control lines:
  - Master-ready,
  - Slave-ready.
- Master-ready signal is asserted by the master to indicate to the slave that it is ready to participate in a data transfer.
- Slave-ready signal is asserted by the slave in response to the master-ready from the master, and it indicates to the master that the slave is ready to participate in a data transfer.

Asynchronous bus (contd..)

- Data transfer using the handshake protocol:
  - Master places the address and command information on the bus.
  - Asserts the Master-ready signal to indicate to the slaves that the address and command information has been placed on the bus.
  - All devices on the bus decode the address.
  - Address slave performs the required operation, and informs the processor it has done so by asserting the Slave-ready signal.
  - Master removes all the signals from the bus, once Slave-ready is asserted.
  - If the operation is a Read operation, Master also strobes the data into its input buffer.

Asynchronous bus (contd..)



Skew occurs when two signals transmitted simultaneously from one source arrive at the destination at different times.

t_o - Master places the address and command information on the bus.

t_1 - Master asserts the Master-ready signal. Master-ready signal is asserted at t_1 instead of t_o

t_2 - Addressed slave places the data on the bus and asserts the Slave-ready signal.

t_3 - Slave-ready signal arrives at the master.

t_4 - Master removes the address and command information.

t_5 - Slave receives the transition of the Master-ready signal from 1 to 0. It removes the data and the Slave-ready signal from the bus.

Asynchronous bus (contd..)

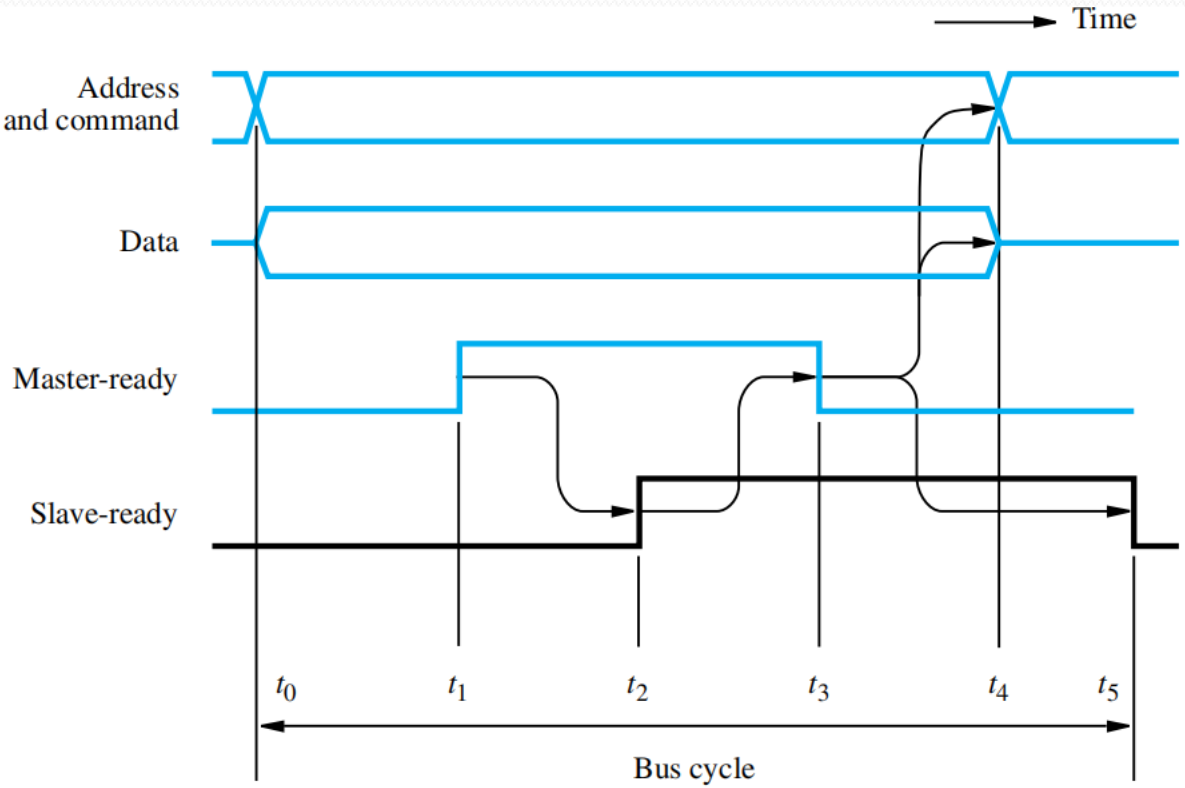


Figure 7.7 Handshake control of data transfer during an output operation.

Asynchronous vs. Synchronous bus

- Advantages of asynchronous bus:
  - Eliminates the need for synchronization between the sender and the receiver.
  - Can accommodate varying delays automatically, using the Slave-ready signal.
- Disadvantages of asynchronous bus:
  - Data transfer rate with full handshake is limited by two-round trip delays.
  - Data transfers using a synchronous bus involves only one round trip delay, and hence a synchronous bus can achieve faster rates.

Arbitration

- There are occasions when two or more entities contend for the use of a single resource in a computer system.
- In such cases, it is necessary to decide which device will access the resource first.
- The decision is usually made in an arbitration process performed by an arbiter circuit.
- The arbitration process starts by each device sending a request to use the shared resource.
- The arbiter associates priorities with individual requests. If it receives two requests at the same time, it grants the use of the slave to the device having the higher priority first.

Arbitration

- A single bus is the shared resource.
- The device that initiates data transfer requests on the bus is the bus master. In case of multiple requests for the shared bus at the same time the decision to select next master is made by an arbiter circuit.
- The arbitration process starts by each device sending a request to use the shared resource.
- The arbiter associates priorities with individual requests. If it receives two requests at the same time, it grants the use of the slave to the device having the higher priority first.

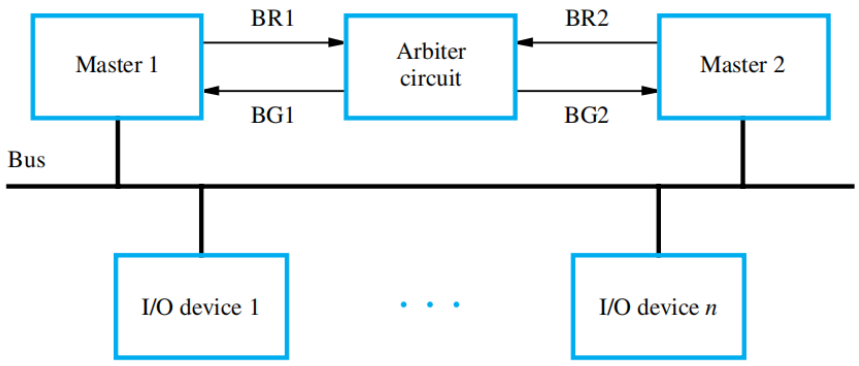


Figure 7.8 Bus arbitration.

Arbitration

- Assume that master 1 has the highest priority, followed by the others in increasing numerical order.
- Master 2 sends a request to use the bus first, the arbiter grants the bus to this master by asserting BG2.
- When master 2 completes its data transfer operation, it releases the bus by deactivating BR2. By that time, both masters 1 and 3 have activated their request lines.
- Note that the bus is granted to master 1 before master 3 even though master 3 activated its request line before master 1.

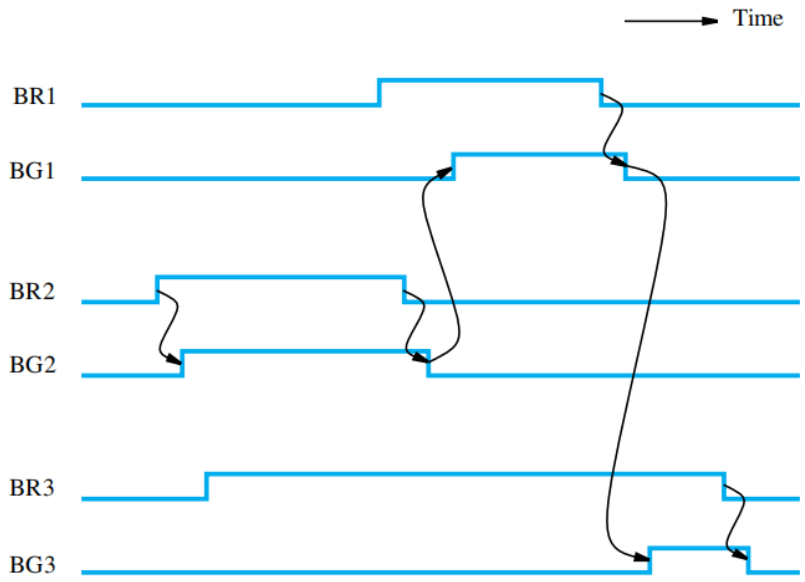


Figure 7.9 Granting use of the bus based on priorities.

Interface Circuits

Interface circuits

- I/O interface consists of the circuitry required to connect an I/O device to a computer bus.
- Side of the interface which connects to the computer has bus signals for:
  - Address,
  - Data
  - Control
- Side of the interface which connects to the I/O device has:
  - Datapath and associated controls to transfer data between the interface and the I/O device.
  - This side is called as a "port".
- Ports can be classified into two:
  - Parallel port,
  - Serial port.

Interface circuits (contd..)

- Parallel port transfers data in the form of a number of bits, normally 8 or 16 to or from the device.
- Serial port transfers and receives data one bit at a time.
- Processor communicates with the bus in the same way, whether it is a parallel port or a serial port.
  - Conversion from the parallel to serial and vice versa takes place inside the interface circuit.

Interface Circuits

- Provides a register for temporary storage of data
- Includes a status register containing status information that can be accessed by the processor
- Includes a control register that holds the information governing the behavior of the interface
- Contains address-decoding circuitry to determine when it is being addressed by the processor
- Generates the required timing signals
- Performs any format conversion that may be necessary to transfer data between the processor and the I/O device, such as parallel-to-serial conversion in the case of a serial port

Parallel Interface

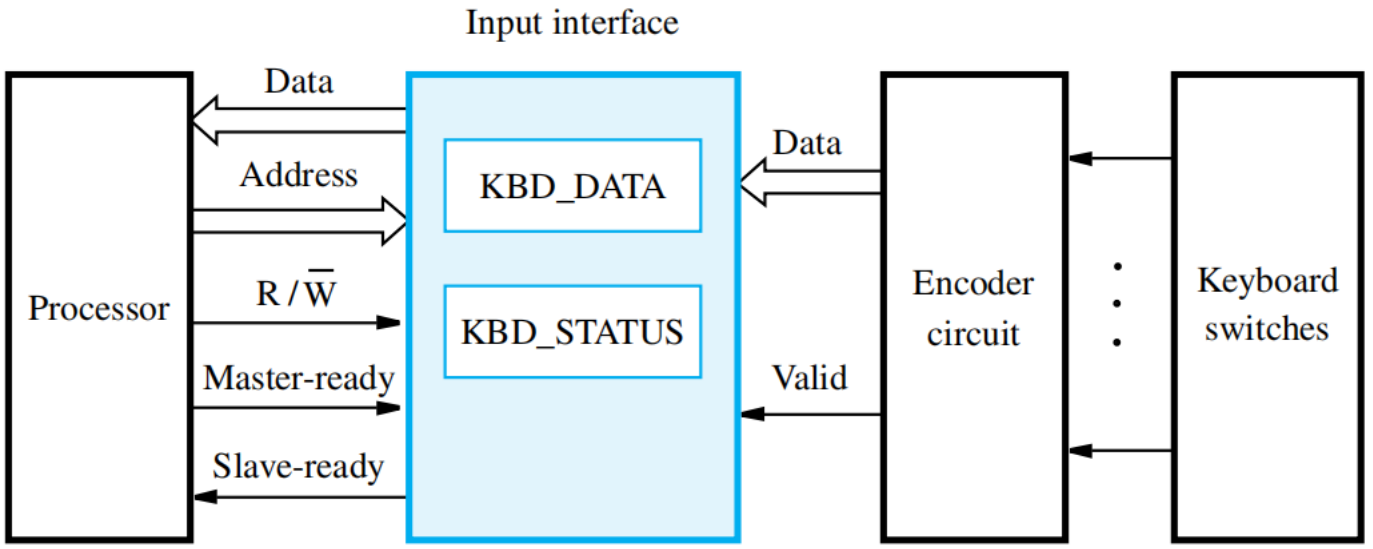


Figure 7.10 Keyboard to processor connection.

- *Processor is 32-bits and uses memory-mapped I/O and the asynchronous bus protocol.*
- *On the processor side of the interface we have:*
  - *Data lines.*
  - *Address lines*
  - *Control or R/W line.*
  - *Master-ready signal and*
  - *Slave-ready signal.*

Parallel Interface (contd..)

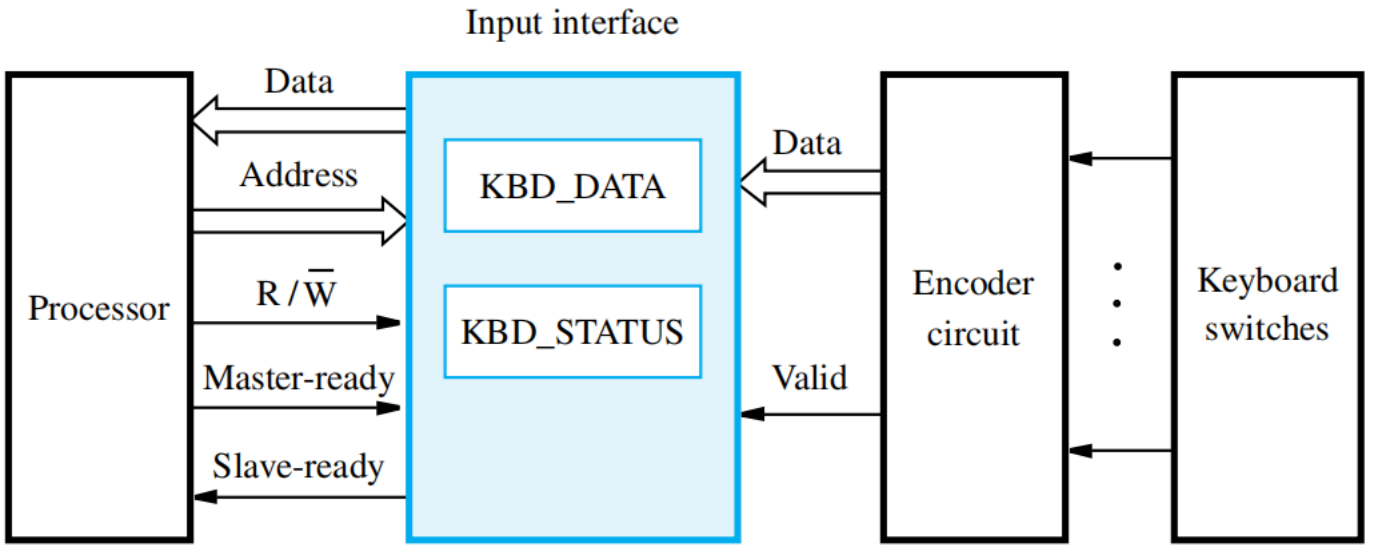
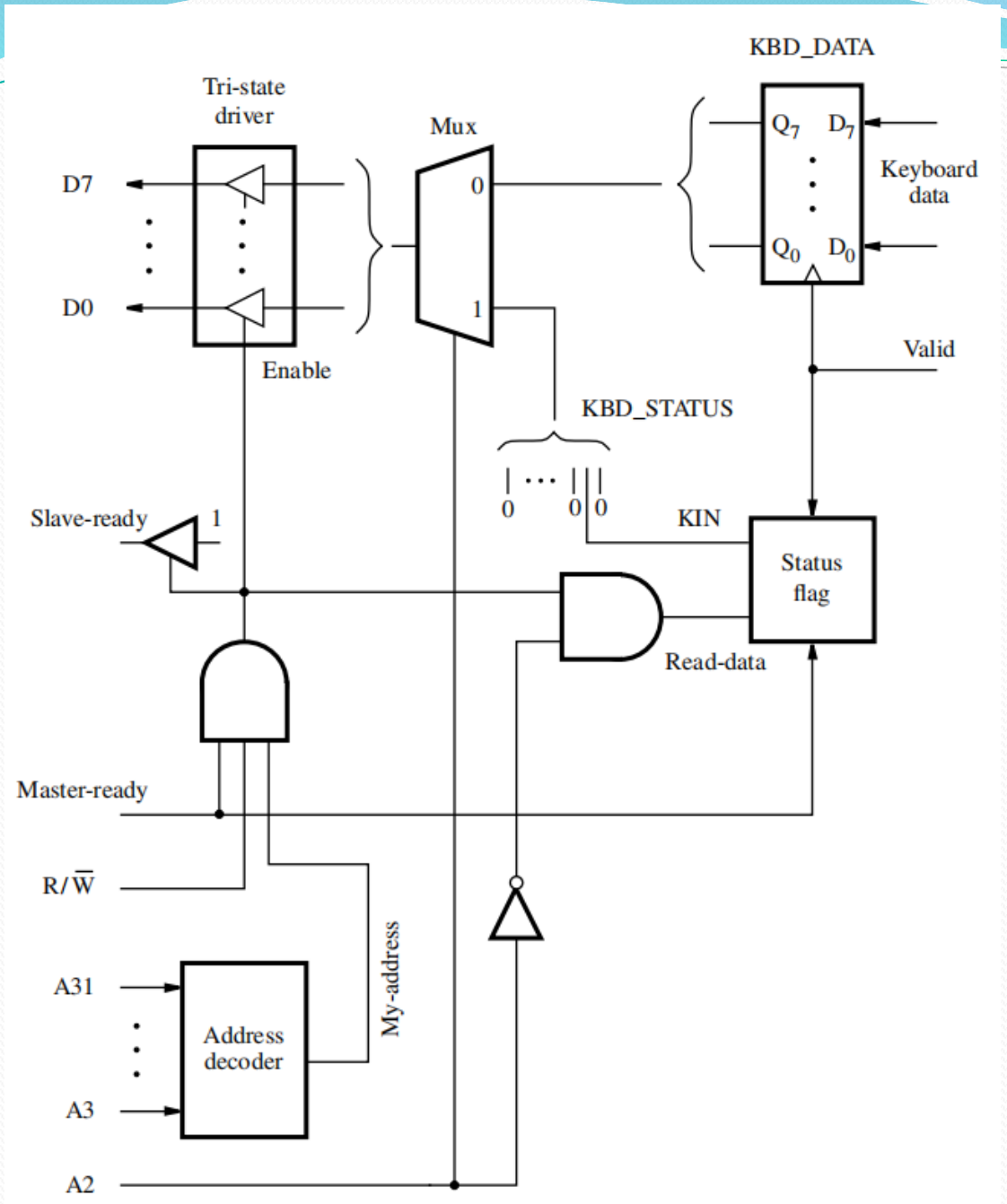


Figure 7.10 Keyboard to processor connection.

- *Encoder circuit which generates a code for the key pressed.*
- *Debouncing circuit which eliminates the effect of a key bounce (a single key stroke may appear as multiple events to a processor).*
- *Data lines contain the code for the key.*
- *Valid line changes from 0 to 1 when the key is pressed. This causes the code to be loaded into KBD_DATA and KIN to be set to 1.*

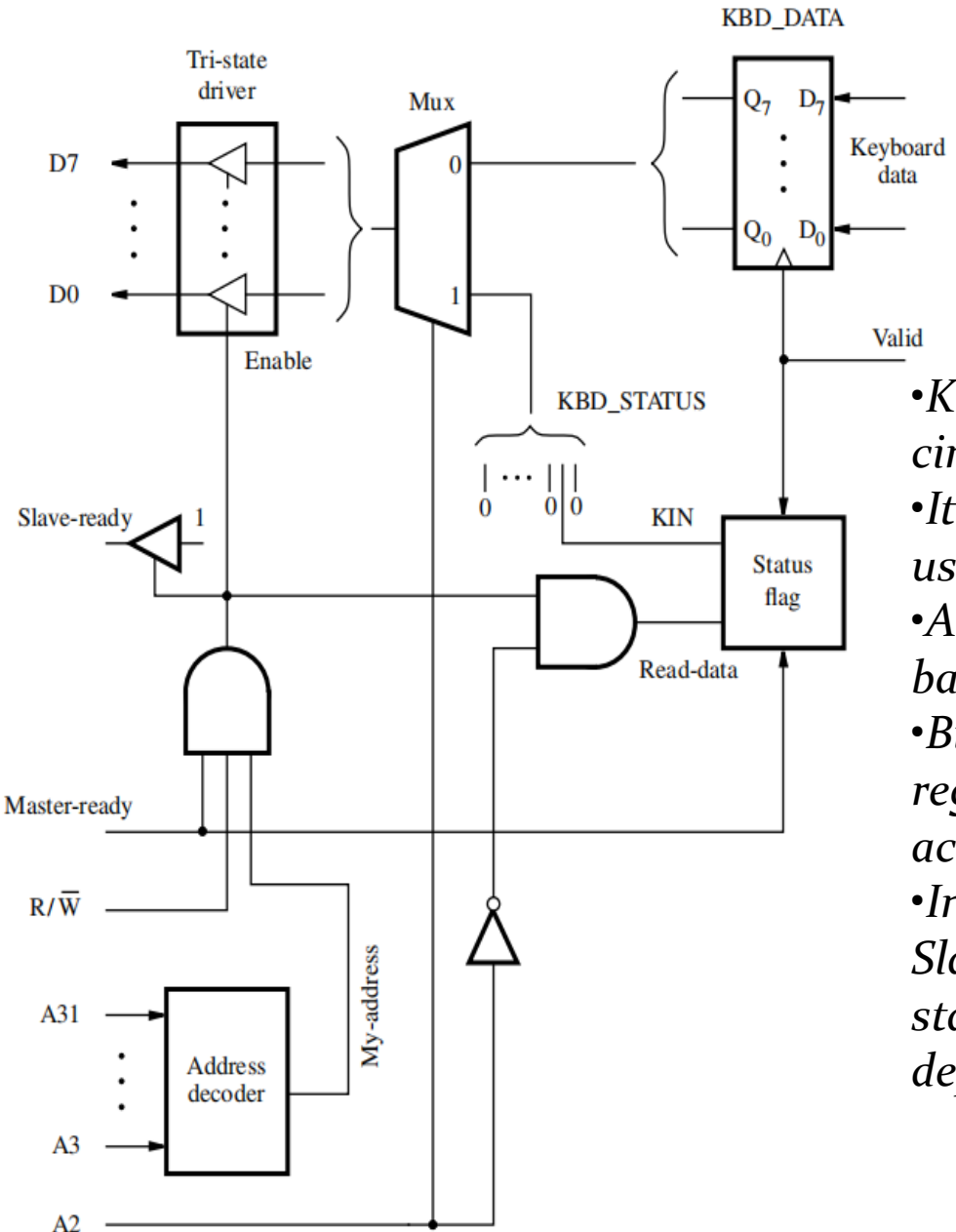


A2=0, read KBD_DATA,
A2=1, read KBD_STATUS

$R/\overline{W}$ *= 1, Read*
=0, Write

Valid = 1, loads data into KBD_DATA and KIN is set to 1

Input Interface Circuit



- *Output lines of KBD_DATA are are connected to the data lines of the bus by means of 3 state drivers*
- *Drivers are turned on when the processor issues a read signal and the address selects this register.*
- *KIN signal is generated using a status flag circuit.*
- *It is connected to line D1 of the processor bus using a three-state driver.*
- *Address decoder selects the input interface based on bits A_3 through A_{31}.*
- *Bit A2 determines whether the status or data register is to be read, when Master-ready is active. A2=0 KBD_DATA, A2=1 KBD_STATUS*
- *In response, the processor activates the Slave-ready signal, when either the Read-status or Read-data is equal to 1, which depends on line A2.*

Parallel port (contd..)

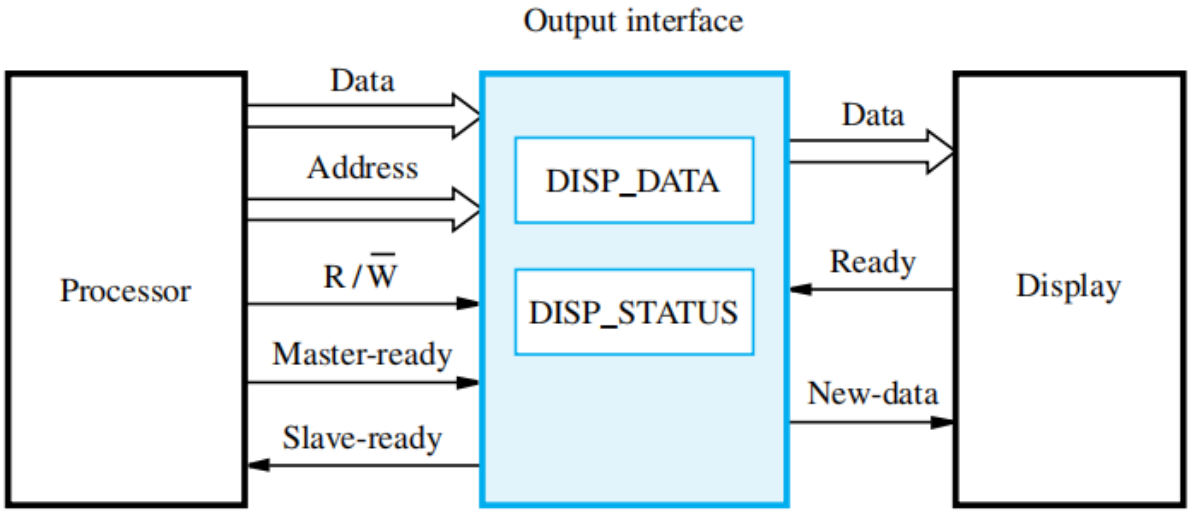


Figure 7.13 Display to processor connection.

- *Printer is connected to a processor using a parallel port.*
- *Processor is 32 bits, uses memory-mapped I/O and asynchronous bus protocol.*
- *On the processor side:*
  - *Data lines.*
  - *Address lines*
  - *Control or R/W line.*
  - *Master-ready signal and*
  - *Slave-ready signal.*

Parallel port (contd..)

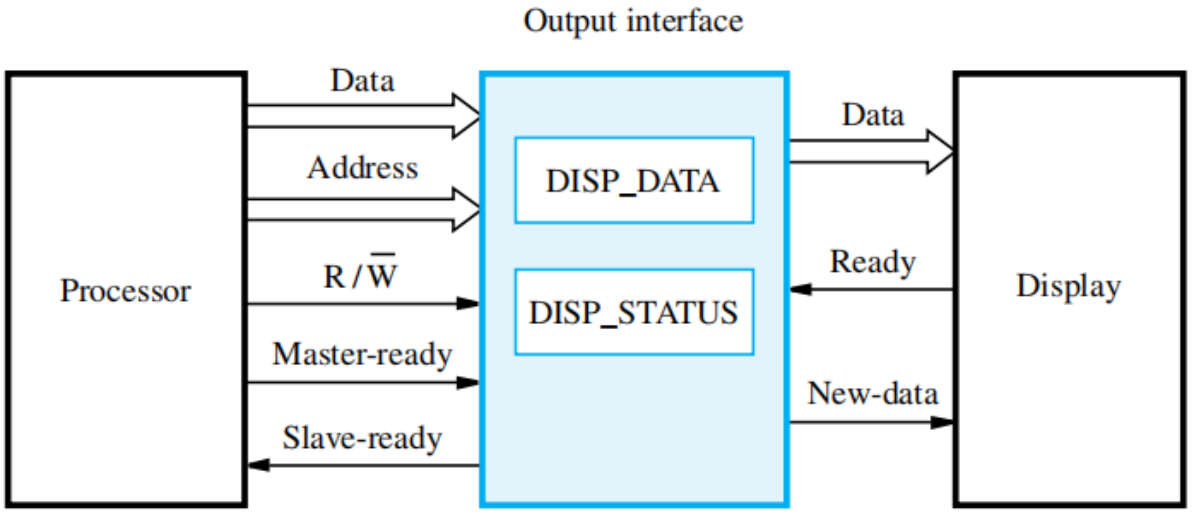
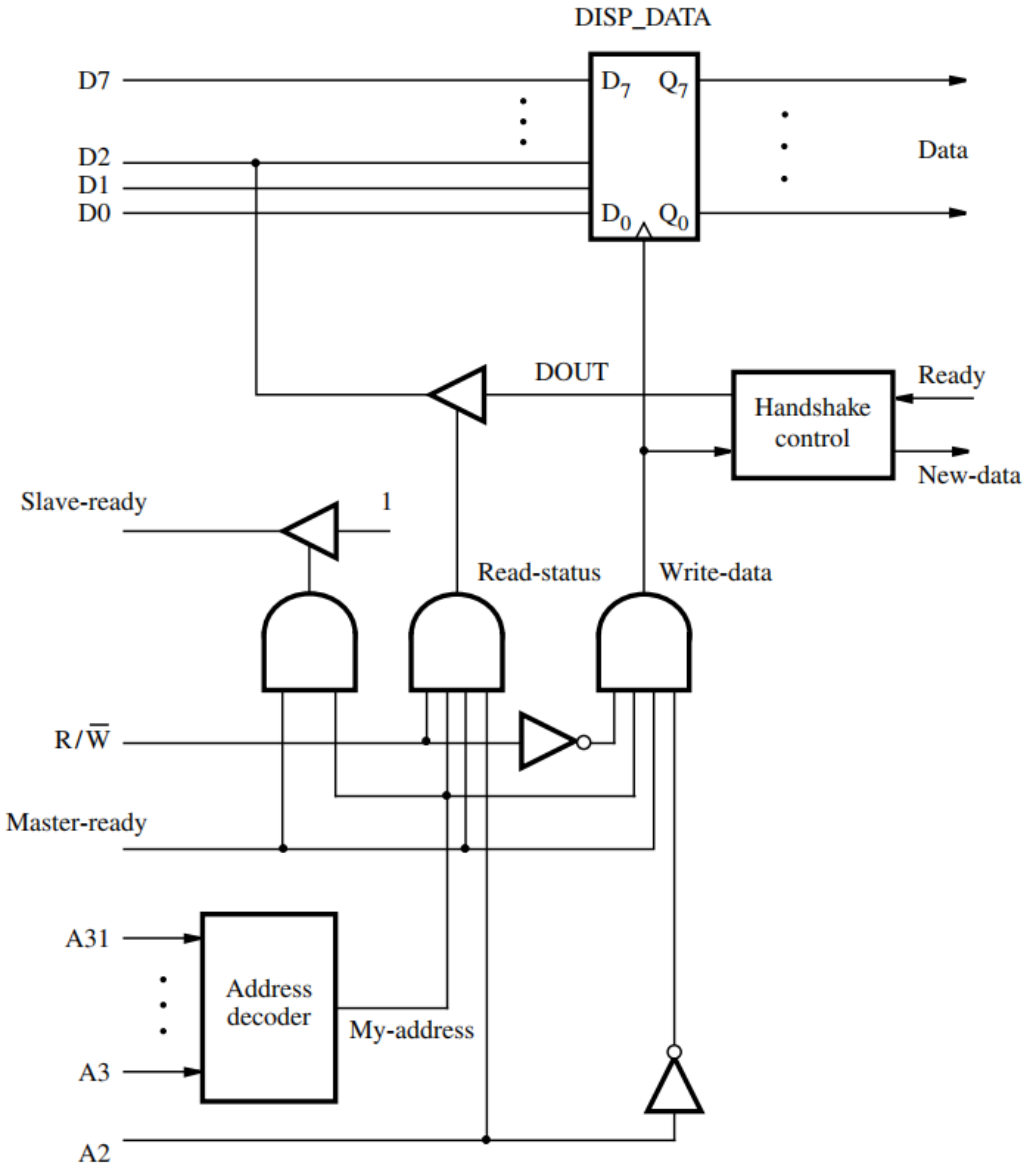


Figure 7.13 Display to processor connection.

- *On the printer side:*
  - Ready *signal line which the printer asserts when it is ready to accept a character. This causes the DOUT flag to be set to 1.*
  - *Processor places a new character into a DISP_DATA register.*
  - *New-data signal, asserted by the interface circuit when it places a new character on the data lines.*

1. *Read DOUT: Address on address bus,* $R/\overline{W}$ *=1, Master ready = 1, A2 = 1, Slve ready=1*
2. *If DOU=1, write data into DISP_DATA: Address on address bus,* $R/\overline{W}$ *=0, Master ready = 1, A2 = 0, Slve ready=1, New-data = 1*

Output Interface Circuit



- *Data lines of the processor bus are connected to the DISP_DATA register of the interface.*
- *The status flag DOUT is connected to the data line D2 using a three-state driver.*
- *The three-state driver is turned on, when the control Read-status line is 1.*
- *Address decoder selects the output interface using address lines A3 through A31.*
- *Address line A2 determines whether the data is to be loaded into the DISP_DATA register or status flag is to be read.*
- *when display is ready, asserts "ready" and sets DOUT to 1.*
- *when DOUT=1, processor writes ch/c into DISP_DATA; clears DOUT; sets "New-data" to 1*
- *in response, display drives "Ready" low; dispalys character*

Serial port

- Serial port is used to connect the processor to I/O devices that require transmission of data one bit at a time.
- Serial port communicates in a bit-serial fashion on the device side and bit parallel fashion on the bus side.
  - Transformation between the parallel and serial formats is achieved with shift registers that have parallel access capability.

Serial Interface

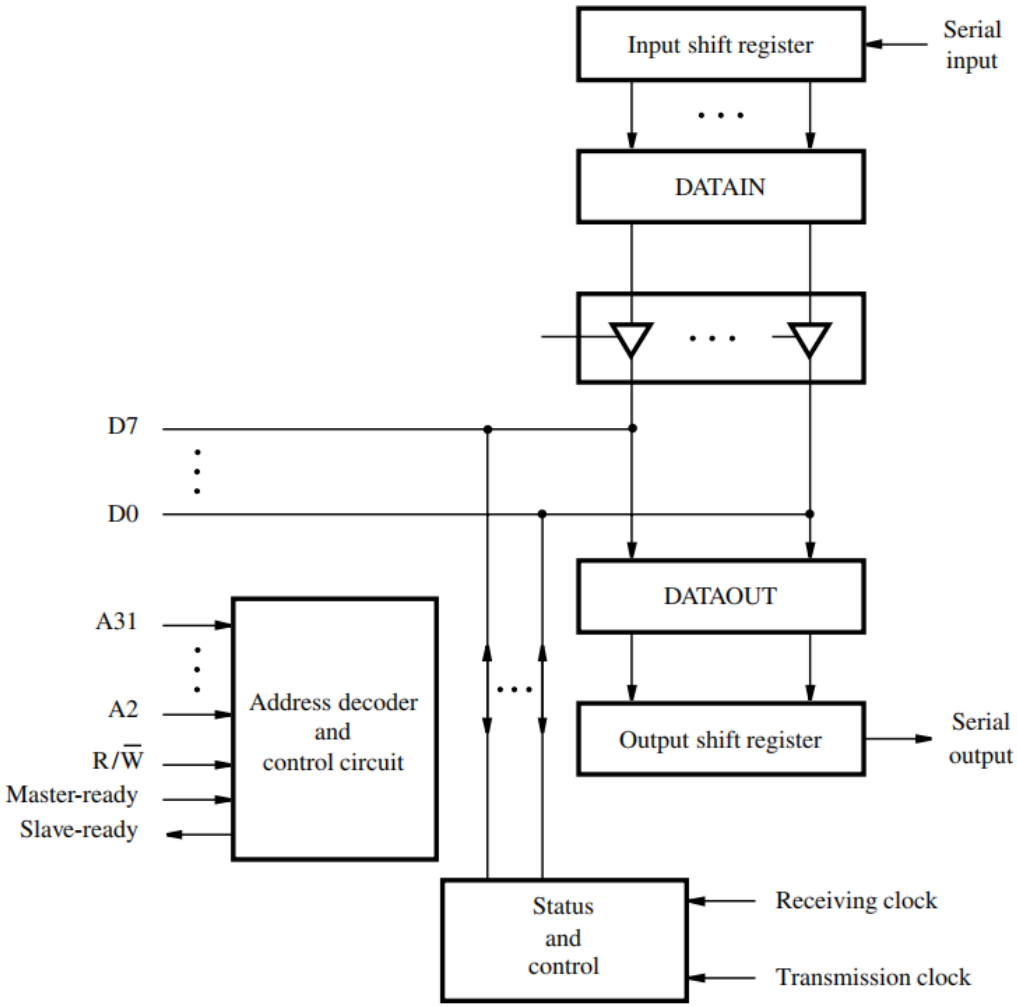


Figure 7.15 A serial interface.

- *Input shift register accepts input one bit at a time from the I/O device.*
- *Once all the 8 bits are received, the contents of the input shift register are loaded in parallel into DATAIN register.*
- *Output data in the DATAOUT register are loaded into the output shift register.*
- *Bits are shifted out of the output shift register and sent out to the I/O device one bit at a time.*
- *As soon as data from the input shift reg. are loaded into DATAIN, it can start accepting another 8 bits of data.*
- *Input shift register and DATAIN registers are both used at input so that the input shift register can start receiving another set of 8 bits from the input device after loading the contents to DATAIN, before the processor reads the contents of DATAIN. This is called as double-buffering.*

Serial Interface

- Two status flags, SIN and SOUT, are maintained by the Status and control block.
- SIN = 1 when new data are loaded into DATAIN from the shift register,
- SIN = 0 when these data are read by the processor.
- SOUT = 0 when the processor writes new data into DATAOUT.
- SOUT = 1 when data are transferred from DATAOUT to the output shift register.

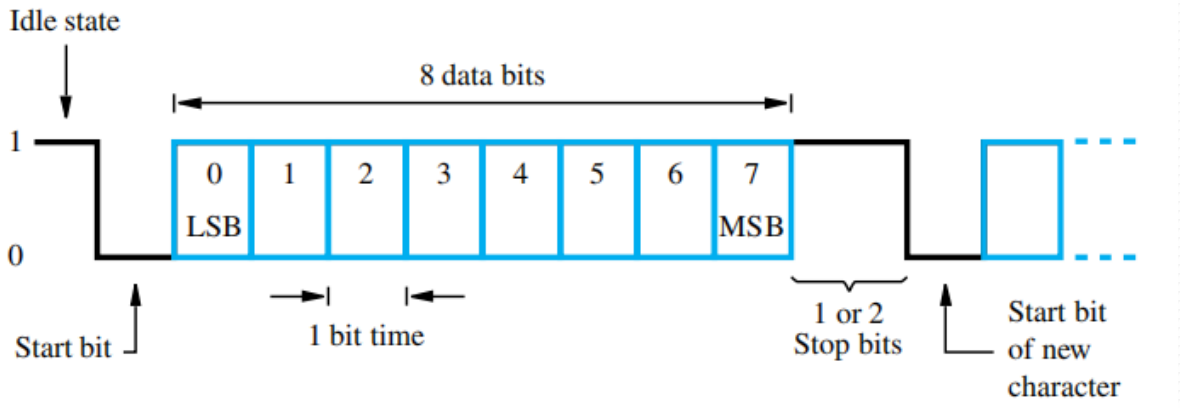
Serial Interface

- Since there is no separate line to carry a clock signal from the transmitter to the receiver, the timing information needed must be embedded into the transmitted data using an encoding scheme. There are two basic approaches.

- asynchronous transmission-the receiver uses a clock that is not synchronized with the transmitter clock.
- synchronous transmission-the receiver is able to generate a clock that is synchronized with the transmitter clock.

Asynchronous Transmission

- Uses a technique called start-stop transmission
- Data are organized in small groups of 6 to 8 bits, with a well-defined beginning and end.
- A character is transmitted as a 0 bit, referred to as the Start bit, followed by 8 data bits and 1 or 2 Stop bits(logic value of 1).
- 1 to 0 start bit t alerts the receiver about data transmission.
- Receiver uses its clock and reads the next 8 bits.
- The Stop bits ensure that the Start bit of the next character will be recognized.
- When no transmission, the line remains in at 1.



Synchronous Transmission

- Asynchronous scheme is useful only where the speed of transmission is sufficiently low and the conditions on the transmission link are such that the square waveforms shown in the figure maintain their shape.
- In synchronous transmission, the receiver generates a clock that is synchronized to that of the transmitter by observing successive 1-to-0 and 0-to-1 transitions in the received signal.
- A variety of encoding schemes are used to ensure that enough signal transitions occur to enable the receiver to generate a synchronized clock and to maintain synchronization.
- Once synchronization is achieved, data transmission can continue indefinitely.
- Data are usually transmitted in large blocks consisting of several hundreds or several thousands of bits.
- The beginning and end of each block are marked by appropriate codes.
- Synchronous transmission enables very high data transfer rates.

Serial port (contd..)

- Serial interfaces require fewer wires, and hence serial transmission is convenient for connecting devices that are physically distant from the computer.
- Speed of transmission of the data over a serial interface is known as the “bit rate”.
  - Bit rate depends on the nature of the devices connected.
- In order to accommodate devices with a range of speeds, a serial interface must be able to use a range of clock speeds.
- Several standard serial interfaces have been developed:
  - Universal Asynchronous Receiver Transmitter (UART) for low-speed serial devices.
  - RS-232-C for connection to communication links.